WireMod Spline Generation Guide for SBND

Alexander Antonakis[1]

[1]University of California, Santa Barbara

September 9, 2026

Contents

1 Introduction & Code Repositories

To produce WireMod splines for SBND, we split the task into two main parts:

1. Generate multi-dimensional histograms with spatial, angular, and hit response variables that have calibration corrections applied.
    - We have an interface for calibration NTuples as well as flat CAF trees.
    - The calibration NTuples allow us to generate splines from through-going muons. These provide a stable dE/dx from the MIP-like cosmic off-beam samples.
    - The flat CAF trees allow us to select protons as a way to probe dE/dx with Wiremod. This population will allow us to study the performance of the WireMod splines extracted from MIP muons when applied to highly ionizing tracks. Additionally, we may be able to add dE/dx as a WireMod dimension.
    - The code repository, `SBNDWireModHists`, can be found here:
        - URL: https://github.com/aantonakis/SBNDWireModHists
2. Extract the data/mc MPV ratios for the hit response variables as a function of the spatial and angular dimensions. We partition the spatial and angular components of the multi-dimensional histograms from step 1 to achieve sufficient track statistics for the MPV extraction. The hit response ratios as a function of the spatial and angular binning define the WireMod splines. These splines typically constitute a family of 2D ratio splines that can leverage interpolation functionaliy from packages such as ROOT.

The spline generation workflow is currently split across two repositories:

- `SBNDWireModSplineGeneration`: Python workflow scripts, ROOT output files, CSV summaries, and diagnostic plots.
    - URL: https://github.com/aantonakis/SBNDWireModSplineGeneration
- `WireMod-Spline-Generation-Guide`: this documentation project.
    - URL: https://github.com/aantonakis/WireMod-Spline-Generation-Guide

The current 2D workflow uses PyROOT for all ROOT I/O and histogram projections, while Matplotlib is used for diagnostic plots. The ROOT environment is loaded through the wrapper script:

```
./scripts/run_with_root.sh <python script> <options>
```

2 WireMod Histogram Generation

2.1 Purpose

The `SBNDWireModHists` repository produces the ROOT `THnSparseD` inputs used by the spline generation workflow. This step reads the selected data or simulation samples, applies the requested calibration corrections and event/track selections, and fills one track-count histogram and one hit-response histogram for each requested wire plane or angular region.

The central Gen2 macro is:

```
macros/multi_dim_tracks_gen2_grid.C
```

For the angle-bin workflow, the corresponding macro is:

```
macros/multi_dim_tracks_gen2_anglebin_grid.C
```

The standard histogram objects are named `hTrackN` and `hHitN`, where N is the wire-plane index. The `hTrackN` histograms are used to count tracks and construct adaptive regions. The `hHitN` histograms store the hit-response axis, for example charge, dQ/dx, or width, together with the selected WireMod coordinates.

2.2 Configuration Files

Histogram axes and selections are controlled by text configuration files under:

`Configs/Gen2`

The configuration file defines the global binning with `Nbins`, `Xmin`, and `Xmax`, while the `dim` field selects which axes are actually stored in the output sparse histograms. For the current angle-bin XYZ workflow, the template configs store:

- `dim: 0 1 2 6` for (x, y, z, Q),
- `dim: 0 1 2 5` for $(x, y, z, dQ/dx)$,
- `dim: 0 1 2 7` for (x, y, z, W).

The angular coordinates are not stored as axes in these reduced output histograms. Instead, they are used as selections. Each generated angle-bin config contains `PlaneIndex`, `UseAbsAngles`, `ThetaXWMin`, `ThetaXWMax`, `ThetaYZMin`, `ThetaYZMax`, `AngleBinRegion`, and `AngleBinTracks`. These fields select a single folded angular region for a single wire plane.

The current templates are:

```
Configs/Gen2/config_xyz_spline_anglebin_template_q.cfg
Configs/Gen2/config_xyz_spline_anglebin_template_dq.cfg
Configs/Gen2/config_xyz_spline_anglebin_template_w.cfg
```

2.3 Generating Angle-Bin Configs

The angle-bin configs are generated from folded 2D angular-bin CSV files produced by the angular binning workflow. The helper script is:

`scripts/generate_gen2_anglebin_configs.py`

It reads the per-plane angular bins, chooses the appropriate observable template, and writes one config per angular region:

```
python scripts/generate_gen2_anglebin_configs.py --dim q
python scripts/generate_gen2_anglebin_configs.py --dim dq
python scripts/generate_gen2_anglebin_configs.py --dim w
```

The generated configs are stored in observable-specific directories:

```
Configs/Gen2/AngleBins_q/PlaneN
Configs/Gen2/AngleBins_dq/PlaneN
Configs/Gen2/AngleBins_w/PlaneN
```

The current folded angular binning gives 27 regions for planes 0, 2, 4, and 5, and 26 regions for planes 1 and 3. The same angular-region definitions are used for the `q`, `dq`, and `w` observable configs.

2.3.1 Switching Data and MC Configs

The generated angle-bin configs contain an `isData` field that controls whether the histogram-generation macro treats the input as data or simulation. After generating the configs, this field can be changed in bulk with:

`scripts/set_config_isdata.py`

For example, to set all `q` angle-bin configs to simulation mode:

```
python scripts/set_config_isdata.py Configs/Gen2/AngleBins_q --value false
```

and to set them back to data mode:

```
python scripts/set_config_isdata.py Configs/Gen2/AngleBins_q --value true
```

The same command can be applied to the other observable-specific config directories, such as `Configs/Gen2/AngleBins_dq` or `Configs/Gen2/AngleBins_w`. To update every angle-bin config directory under `Configs/Gen2` at once, use:

```
python scripts/set_config_isdata.py --all-anglebins --value false
python scripts/set_config_isdata.py --all-anglebins --value true
```

The `--dry-run` option can be added to either command to print the files that would be changed without editing them.

2.4 Submitting Grid Jobs

The plane-level grid wrapper submits the Gen2 angle-bin macro once for each config in a selected plane and observable directory:

```
grid/submit_multi_dim_tracks_gen2_anglebin_plane.py
```

It loops over the relevant config files and forwards the common grid options to:

```
grid/submit_multi_dim_tracks_gen2_anglebin.py
```

A representative submission command is:

```
python grid/submit_multi_dim_tracks_gen2_anglebin_plane.py \
  --plane 0 \
  --observable q \
  --sample data \
  --tag gen2 \
  -l input_file_list.txt \
  -nfile 100 \
  -ngrid 20
```

For each config, the wrapper builds an output prefix of the form:

```
gen2_<data-or-mc>_<config-name-without-.cfg>
```

For example, the first plane-0 charge angular region is labeled:

```
gen2_data_config_xyz_spline_anglebin_q_plane0_region001
```

The resulting ROOT files are then moved into the spline-generation repository under separate data and MC directories, for example:

```
AngleData/data/plane0
AngleData/mc/plane0
```

3 Simple 2D Spline Generation

3.1 Goal

The first 2D spline production tests use lower-dimensional Gen2 histograms where the first two THnSparse axes are WireMod coordinates or angles, and the third axis is the hit-level response variable. The current samples are:

- `YZQ`: adaptive bins in (y, z), hit response axis Q.
- `YZW`: same (y, z) binning as `YZQ`, hit response axis W.
- `XTXZQ`: adaptive bins in (x, θ_{xw}), hit response axis Q.
- `XTXZW`: same (x, θ_{xw}) binning as `XTXZQ`, hit response axis W.

For each plane, the workflow constructs adaptive 2D regions from the data track-count histogram, extracts data and simulation hit-response distributions in each region, estimates the most probable hit-response scale with an iterative truncated mean, and stores the ratio

$$R_{\mathrm{WireMod}}(a, b) = \frac{\mathrm{MPV}_{\mathrm{data}}(a, b)}{\mathrm{MPV}_{\mathrm{MC}}(a, b)} \tag{1}$$

as a `TGraph2DErrors`. Here (a, b) denotes the two WireMod dimensions used for the file, for example (y, z) or (x, θ_{xw}).

3.2 Input Files

The Gen2 2D input files are stored in:

`/Users/alexanderantonakis/Documents/Codex/2026-06-08/Gen2_2D`

Each file contains one `hTrackN` and one `hHitN` object for each of the six wire planes. The `hTrackN` histograms define the binning statistics. The corresponding `hHitN` histograms provide the hit-response distributions used to extract the data/MC ratio.

3.3 Adaptive Binning

The adaptive binning is produced with `scripts/make_adaptive_binning_2d.py`. The default target is currently 5×10^5 tracks per 2D region. The script projects the 3D `hTrackN` sparse onto axes 0 and 1, converts the projection into a NumPy count matrix, and recursively partitions this plane.

The recursive splitting rule is:

1. Start from the full active 2D plane.
2. Try a quadrant split. If all four quadrants exceed the target count threshold, keep splitting each quadrant.
3. If the quadrant split is not allowed by the threshold, try a binary split along the selected fallback axis.
4. Keep the split only if both children exceed the threshold.
5. Otherwise, keep the current region as a terminal adaptive bin.

For `XTXZ` files, one side of the detector is mostly empty for each plane. The workflow therefore applies an active-x selection before adaptive splitting. The default `auto` behavior uses $x < 0$ for planes 0–2 and $x > 0$ for planes 3–5. This prevents the empty side of the histogram from forcing the algorithm into one overly large bin.

A typical command is:

```
./scripts/run_with_root.sh scripts/make_adaptive_binning_2d.py \
  --input ../Gen2_2D/Wire_Gen2_DATA_YZQ_26Aug2026_Full.root \
  --hist hTrack0 \
  --target 500000 \
  --max-plots 1
```

For the width response files, the same binning is reused from the corresponding charge file: `YZQ` binning is used for `YZW`, and `XTXZQ` binning is used for `XTXZW`.

Table 1: Adaptive 2D region counts from the current Gen2 data track histograms. The `XTXZQ` counts include the active-x side selection.

Plane	`YZQ` regions	`XTXZQ` regions
0	2145	826
1	2129	796
2	1056	913
3	2193	852
4	2140	862
5	1116	912

3.4 Ratio Extraction

The ratio extraction is performed with `scripts/make_2d_ratio_splines.py`. For each adaptive region, the script applies the region as an axis-0/axis-1 selection on the data and MC `hHitN` sparse histograms, projects the response axis, and computes the iterative truncated mean for the data and MC projected distributions.

The truncated-mean settings currently match the earlier 3D spline workflow:

- lower truncation: -2.0 RMS from the current mean,
- upper truncation: $+1.75$ RMS from the current mean,
- convergence tolerance: 10^{-4},
- maximum iterations: 100.

The central spline point is placed at the geometric center of each adaptive region. By default, the script also duplicates points at boundary edges and corners. These edge-augmented points help constrain interpolation near the boundary of the valid 2D phase space. The output ROOT file contains:

- `data_mpv_N`: data truncated-mean MPV graph,
- `mc_mpv_N`: MC truncated-mean MPV graph,
- `splines_N`: data/MC ratio graph.

A typical ratio command is:

```
./scripts/run_with_root.sh scripts/make_2d_ratio_splines.py \
  --data ../Gen2_2D/Wire_Gen2_DATA_YZQ_26Aug2026_Full.root \
  --mc ../Gen2_2D/Wire_Gen2_MC_YZQ_26Aug2026_Full.root \
  --hit-hist hHit0 \
  --idx 0 \
  --binning-label YZQ_26Aug2026
```

For `YZW`, the output label and binning label are different:

```
./scripts/run_with_root.sh scripts/make_2d_ratio_splines.py \
  --data .../Wire_Gen2_DATA_YZW_26Aug2026_Full.root \
  --mc .../Wire_Gen2_MC_YZW_26Aug2026_Full.root \
  --hit-hist hHit0 \
  --idx 0 \
  --output-label YZW_26Aug2026 \
  --binning-label YZQ_26Aug2026
```

3.5 Current Outputs

The current output files are written under:

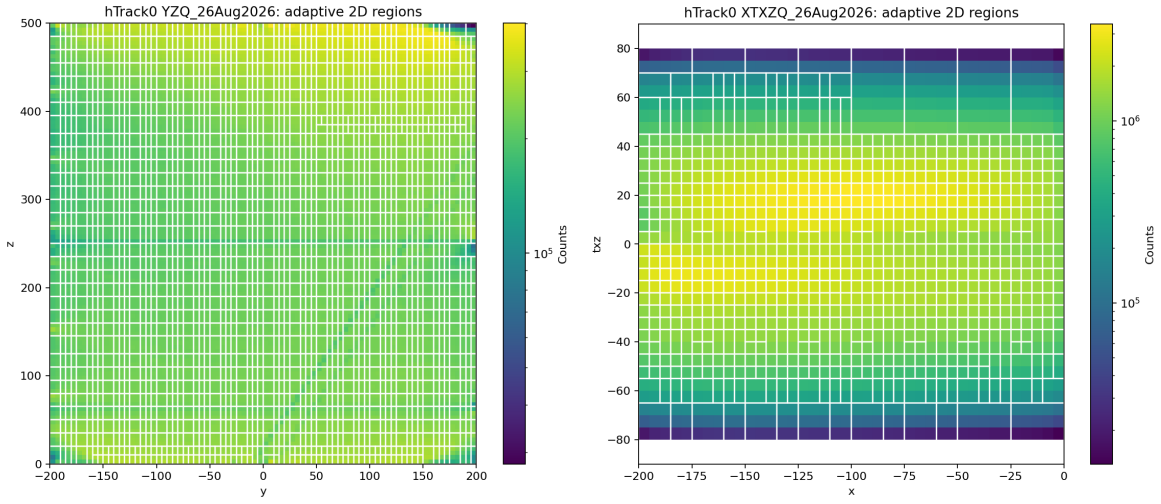
```
plots/adaptive_2d
plots/ratio_splines_2d
```

in the `SBNDWireModSplineGeneration` repository. CSV files store the adaptive regions and MPV ratio summaries, while ROOT files store the final graphs used downstream by WireMod.

Table 2: Current 2D ratio spline point counts. Each entry gives central adaptive-region fits followed by the number of graph points after edge augmentation.

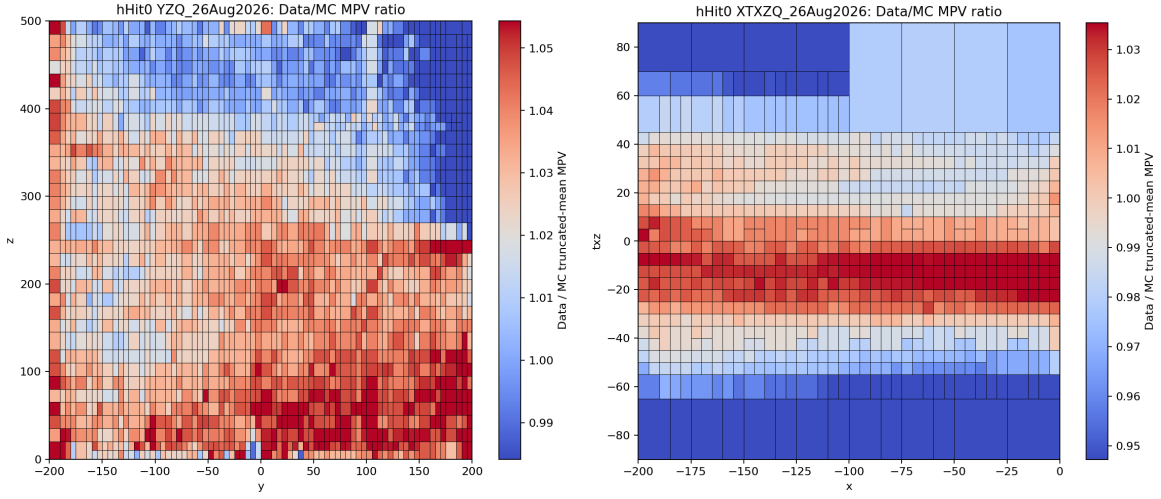
Plane	`YZQ`	`YZW`	`XTXZQ`	`XTXZW`
0	2145 / 2350	2145 / 2350	826 / 897	826 / 897
1	2129 / 2336	2129 / 2336	796 / 864	796 / 864
2	1056 / 1209	1056 / 1209	913 / 996	913 / 996
3	2193 / 2404	2193 / 2404	852 / 920	852 / 920
4	2140 / 2346	2140 / 2346	862 / 930	862 / 930
5	1116 / 1277	1116 / 1277	912 / 990	912 / 990

Figures 1 and 2 show representative outputs for plane 0. The full plane-by-plane diagnostic set is included in Section A.



(a) `YZQ` adaptive regions. (b) `XTXZQ` adaptive regions.

Figure 1: Example adaptive 2D binnings for plane 0. The `XTXZQ` binning uses the active $x < 0$ side for plane 0.



(a) `YZQ` ratio map. (b) `XTXZQ` ratio map.

Figure 2: Example data/MC truncated-mean MPV ratio maps for plane 0.

3.6 Summary Plots

The combined 2D spline ROOT files are also used to make all-plane summary plots for each response observable and coordinate pair. These plots place planes 0–2 in the first row and planes 3–5 in the second row. A fixed color scale from 0.85 to 1.15 is used for all panels so that broad plane-to-plane trends can be compared directly. For the YZ splines, the horizontal axis is z and the vertical axis is y. For the XTXZ splines, the horizontal axis is x and the vertical axis is θ_{xw}.

These summary plots are useful as a common-scale overview of the current 2D results. The appendix figures in Section A provide the complementary per-spline diagnostic view, where the colorbar range is allowed to follow the natural range of each individual 2D spline.

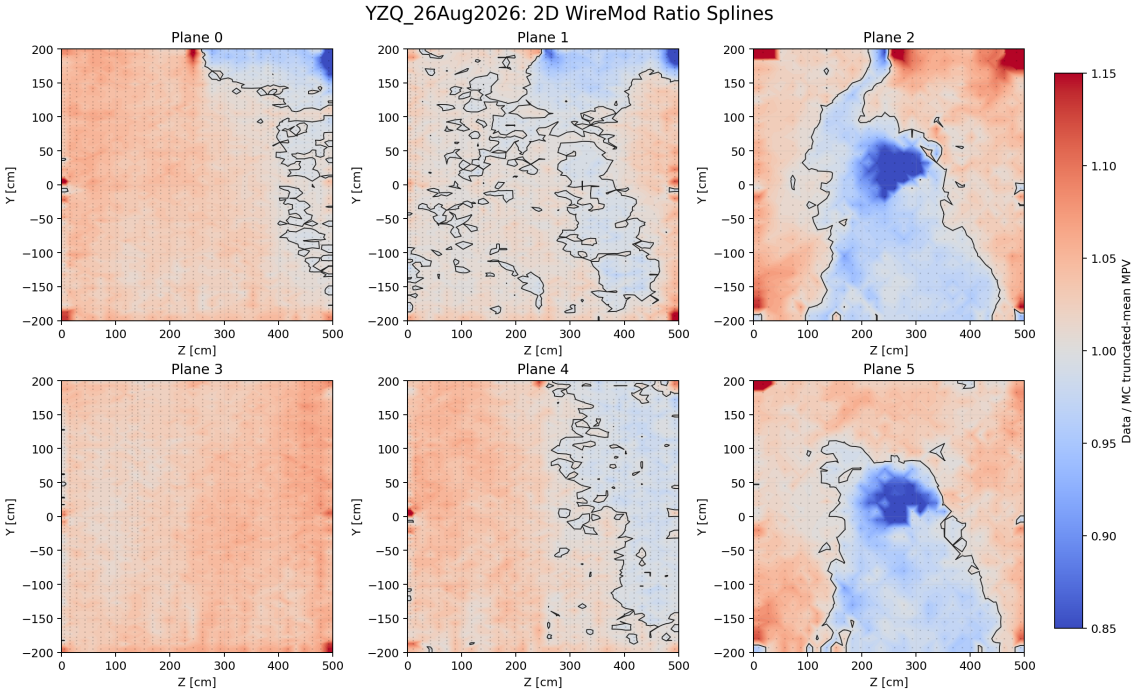


Figure 3: All-plane summary of the `YZQ` data/MC MPV ratio splines. The color scale is fixed from 0.85 to 1.15.

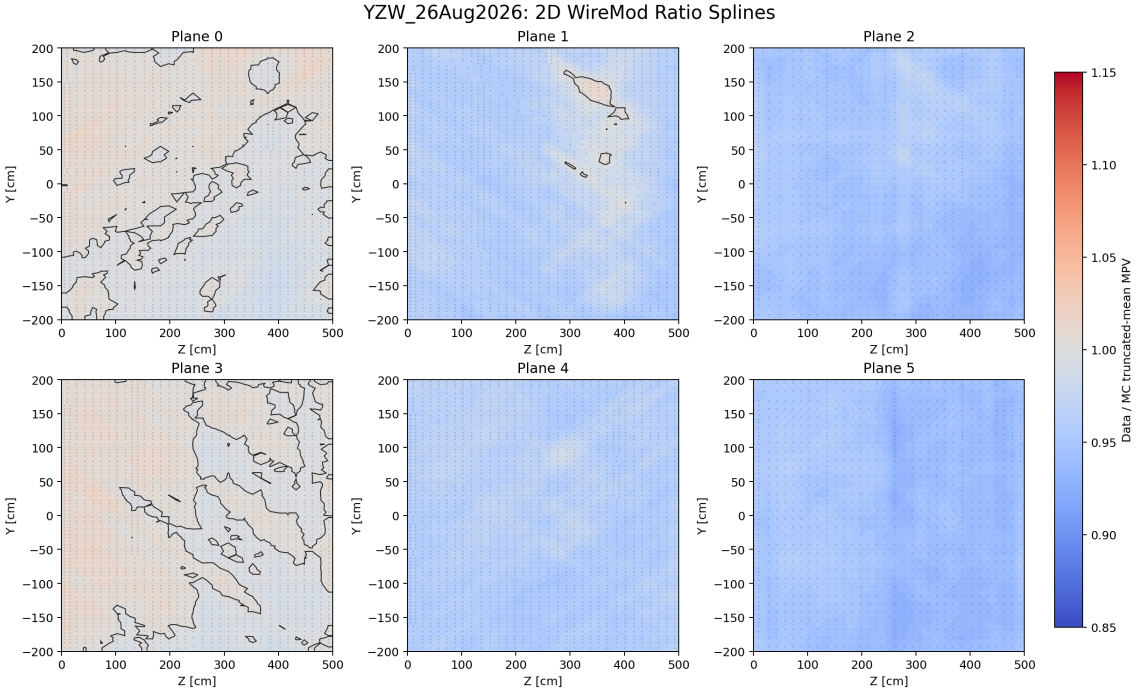


Figure 4: All-plane summary of the `YZW` data/MC MPV ratio splines. The color scale is fixed from 0.85 to 1.15.

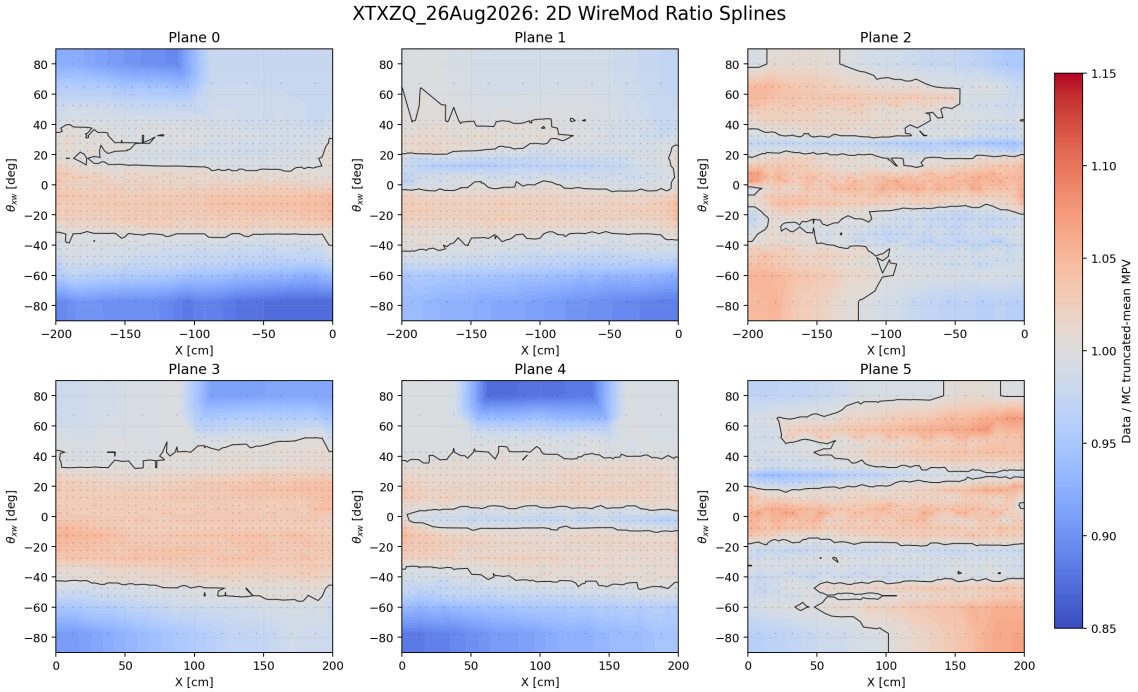


Figure 5: All-plane summary of the `XTXZQ` data/MC MPV ratio splines. The color scale is fixed from 0.85 to 1.15.

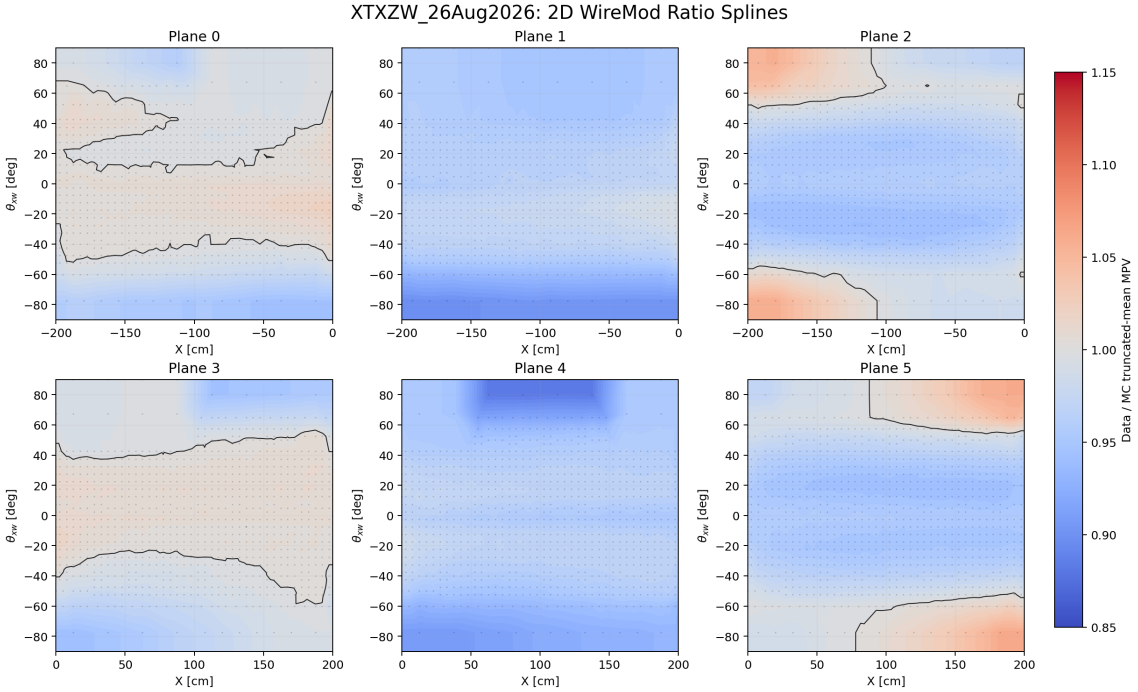


Figure 6: All-plane summary of the `XTXZW` data/MC MPV ratio splines. The color scale is fixed from 0.85 to 1.15.

For the x versus θ_{xw} splines, an alternate symmetric-angle binning was also produced. In this version, the adaptive binning is learned in the $\theta_{xw} > 0$ half-plane and then mirrored to $\theta_{xw} < 0$. This preserves a symmetric angular segmentation while still allowing the positive-angle side to follow the observed track density.

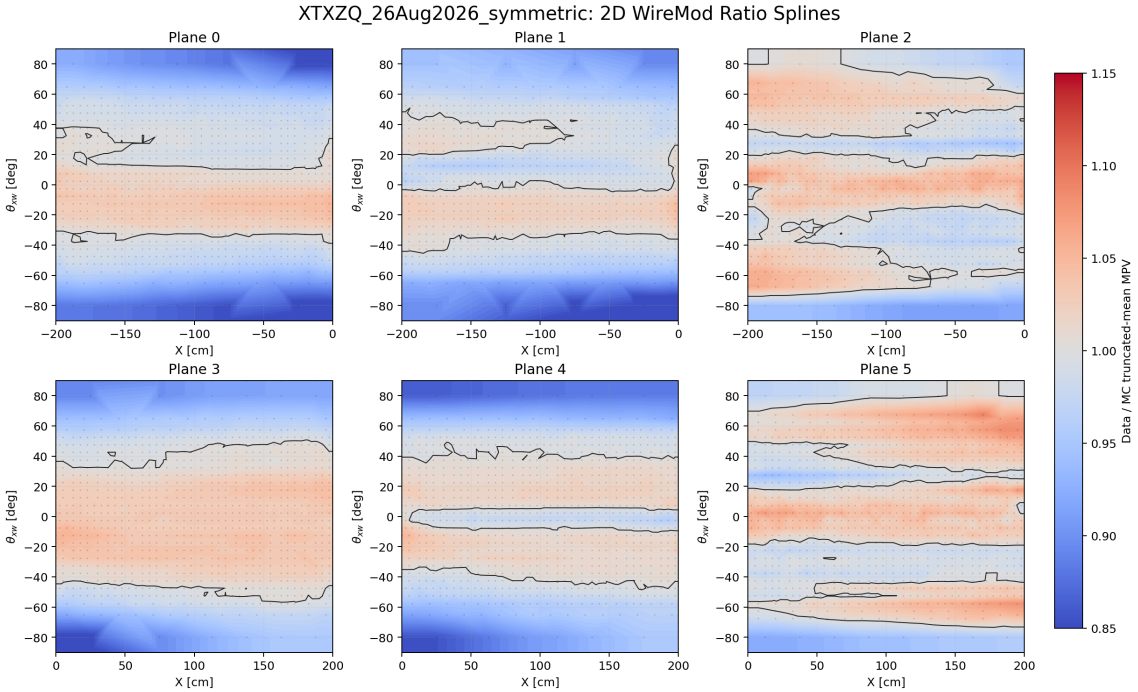


Figure 7: All-plane summary of the symmetric-angle `XTXZQ` data/MC MPV ratio splines. The adaptive binning is learned for $\theta_{xw} > 0$ and mirrored to $\theta_{xw} < 0$. The color scale is fixed from 0.85 to 1.15.

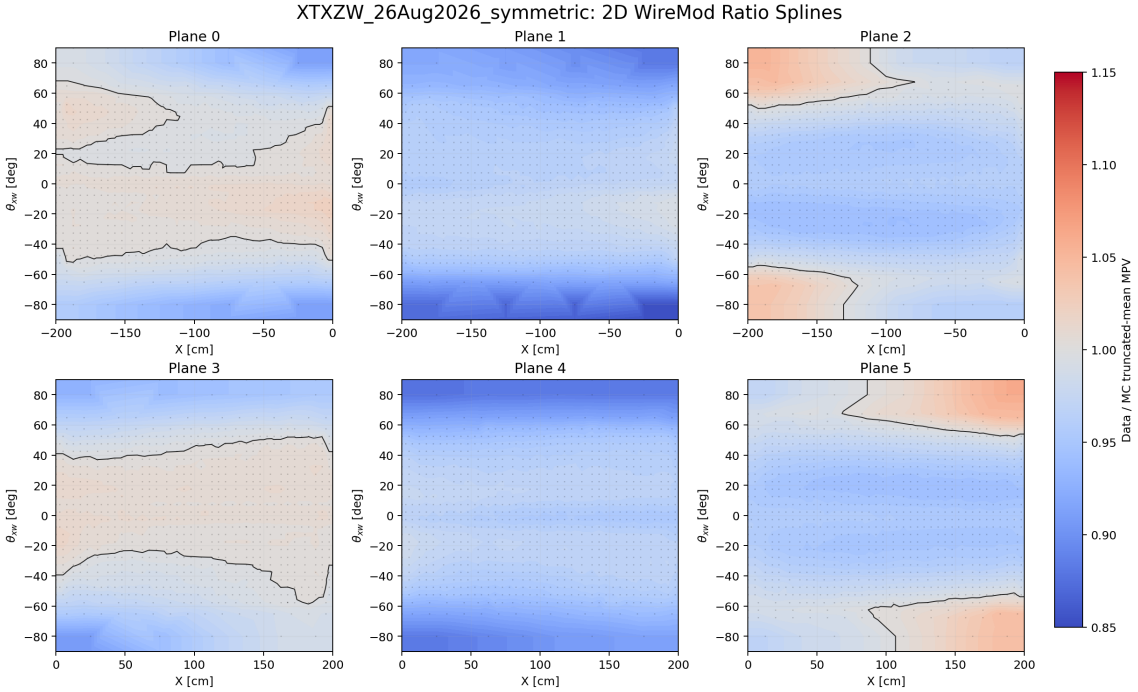


Figure 8: All-plane summary of the symmetric-angle `XTXZW` data/MC MPV ratio splines. The adaptive binning is learned for $\theta_{xw} > 0$ and mirrored to $\theta_{xw} < 0$. The color scale is fixed from 0.85 to 1.15.

4 Higher Dimensional Angle-Binned Spline Generation

4.1 Workflow

The higher-dimensional workflow keeps the angular dependence by first partitioning the two-dimensional angular plane, then producing separate $(x, y, z, \text{response})$ histograms for each angular region. Within a fixed plane and angular region, the workflow proceeds as:

1. Select a folded 2D angular bin in $(|\theta_{xw}|, |\theta_{yz}|)$.
2. Generate data and MC `hTrackN`/`hHitN` histograms in `SBNDWireModHists` using the matching angle-bin config.
3. Use the data `hTrackN` histogram to define adaptive YZ regions independently in each x-bin.
4. In each x-bin and YZ region, project the data and MC `hHitN` histograms onto the response axis.
5. Extract the data and MC truncated-mean MPVs and store the data/MC ratio as a 2D spline in the YZ plane.

The final product for one plane and one angular bin is therefore a family of 2D YZ ratio splines, one spline for each populated x-slice. The full WireMod correction can then be viewed as a set of 2D YZ surfaces indexed by plane, angular bin, x-bin, and response observable.

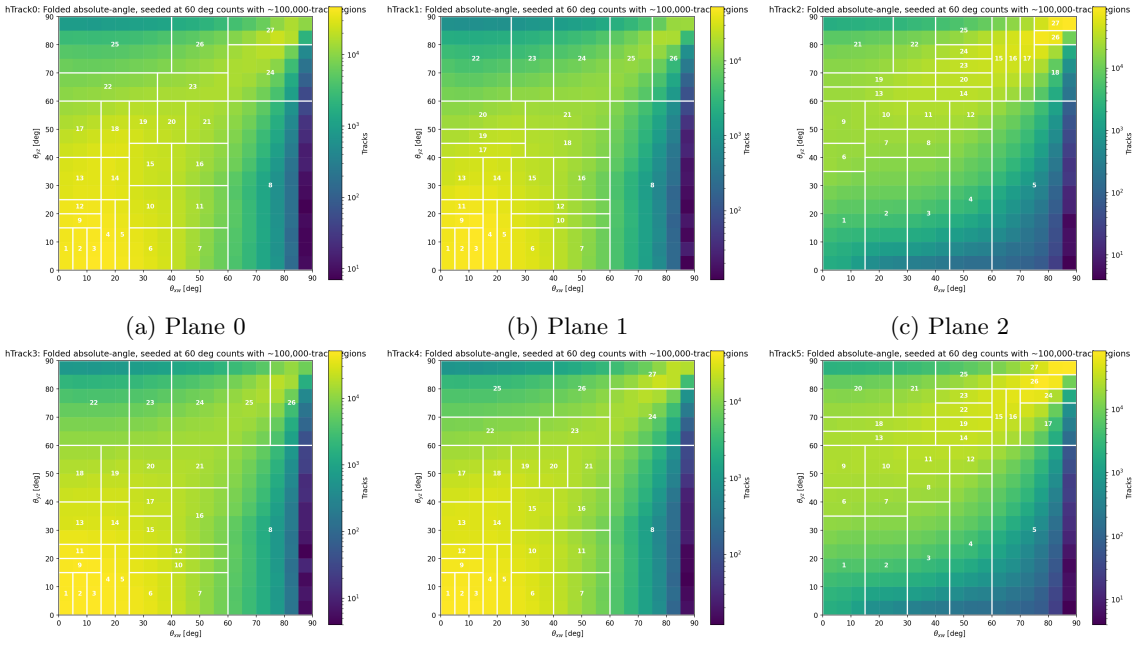
4.2 Angular Binning

The current angular binning uses folded absolute angles, motivated by the approximate symmetry of the hit-response behavior about $\theta = 0$ for both angular variables. The folded angular plane covers:

$$0 \leq |\theta_{xw}| \leq 90^\circ, \qquad 0 \leq |\theta_{yz}| \leq 90^\circ. \tag{2}$$

The binning algorithm first separates the angular space into low-low, low-high, high-low, and high-high regions, with the high-angle boundary at approximately 60°. Adaptive splitting is then run within these regions so that high-angle regions, where stronger gradients are expected, are not swallowed by very large low-angle bins. The current target used for this angular binning is 10^5 tracks per angular region.

Figure 9 shows the folded angular distributions and selected WireMod angular regions for each of the six wire planes.



(a) Plane 0 (b) Plane 1 (c) Plane 2

(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 9: Folded 2D angular track distributions and selected WireMod angular regions. The angular plane is folded into $(|\theta_{xw}|, |\theta_{yz}|)$ and the adaptive binning prioritizes the high-angle regions above approximately 60°.

4.3 Adaptive YZ Binning in X Slices

The x/YZ adaptive binning is produced with:

`scripts/make_adaptive_binning_xyz.py`

This script expects the reduced four-dimensional sparse output from `SBNDWireModHists`, with axes corresponding to $(x, y, z, \text{response})$. For each physical x-bin, it projects the data `hTrackN` histogram onto the YZ plane. If the x-bin has enough tracks, the YZ plane is adaptively split with the same quadrant-first logic used by the simple 2D workflow.

A representative command is:

```
DATA=AngleData/data/plane0
./scripts/run_with_root.sh scripts/make_adaptive_binning_xyz.py \
  --input $DATA/gen2_data_config_xyz_spline_anglebin_q_plane0_region001_full.root \
  --hist hTrack0 \
  --target 100000 \
  --max-plots 5
```

The output label is inferred from names such as `anglebin_q_plane0_region001`, so each angular-region binning is kept separate. The output files are written to:

`plots/adaptive_xyz`

4.4 XYZ Ratio Spline Extraction

The higher-dimensional ratio extraction is performed with:

`scripts/make_xyz_ratio_splines.py`

For each x-bin and adaptive YZ region, the script projects the data and MC `hHitN` histograms onto the response axis and applies the same iterative truncated-mean MPV extraction used in the simple 2D workflow. The output ROOT file contains one set of graphs per x-bin:

- `data_mpv_N_X`: data MPV graph in the YZ plane,
- `mc_mpv_N_X`: MC MPV graph in the YZ plane,
- `splines_N_X`: data/MC ratio graph in the YZ plane.

Here N is the plane index and X is the x-bin index.

A representative command for charge is:

```
DATA=AngleData/data/plane0
MC=AngleData/mc/plane0
./scripts/run_with_root.sh scripts/make_xyz_ratio_splines.py \
  --data $DATA/gen2_data_config_xyz_spline_anglebin_q_plane0_region001_full.root \
  --mc $MC/gen2_mc_config_xyz_spline_anglebin_q_plane0_region001_full.root \
  --hit-hist hHit0 \
  --idx 0
```

For `dq` or `w`, the same charge-derived adaptive YZ binning can be reused with:

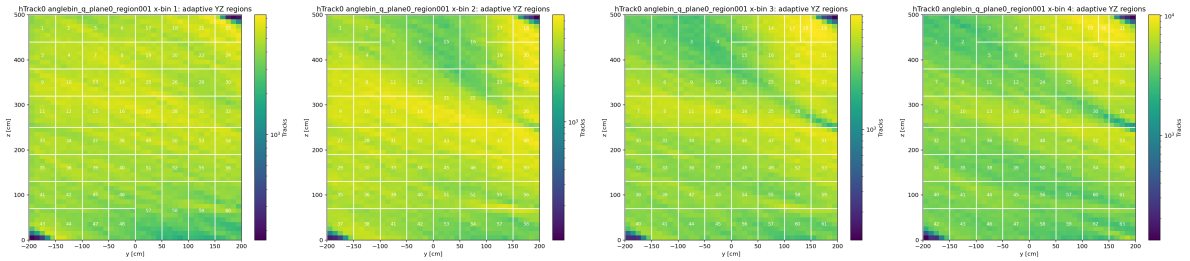
`--binning-observable q`

This keeps the spatial binning fixed across response observables for the same plane and angular region.

4.5 Current Higher-Dimensional Tests

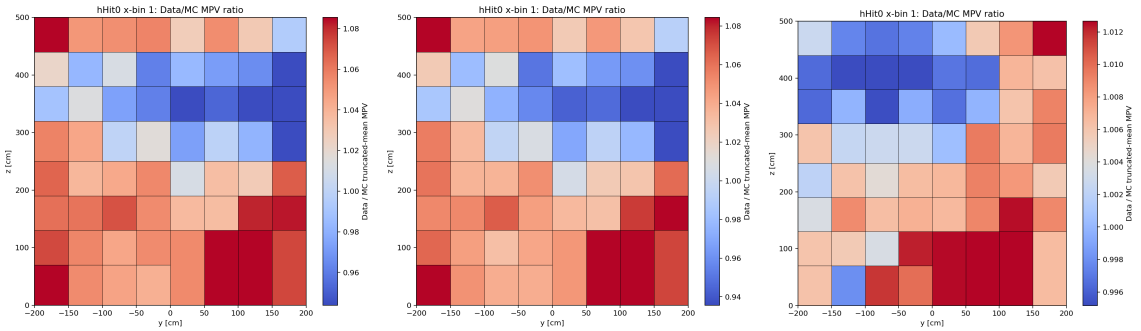
The current end-to-end higher-dimensional test has been run for plane 0, angular region 001, and the `q`, `dq`, and `w` observables. Diagnostic outputs include adaptive YZ region plots for individual x-bins, YZ ratio maps for each x-bin, and optional per-region hit-response plots with data and MC MPV markers.

Figures 10 and 11 show representative plane-0, region-001 outputs. These are useful debugging plots for checking that the YZ binning follows the data track density and that the extracted ratios vary smoothly within each x-slice.



(a) x-bin 1 (b) x-bin 2 (c) x-bin 3 (d) x-bin 4

Figure 10: Example adaptive YZ regions for plane 0, angular region 001, in the first four x-bins.

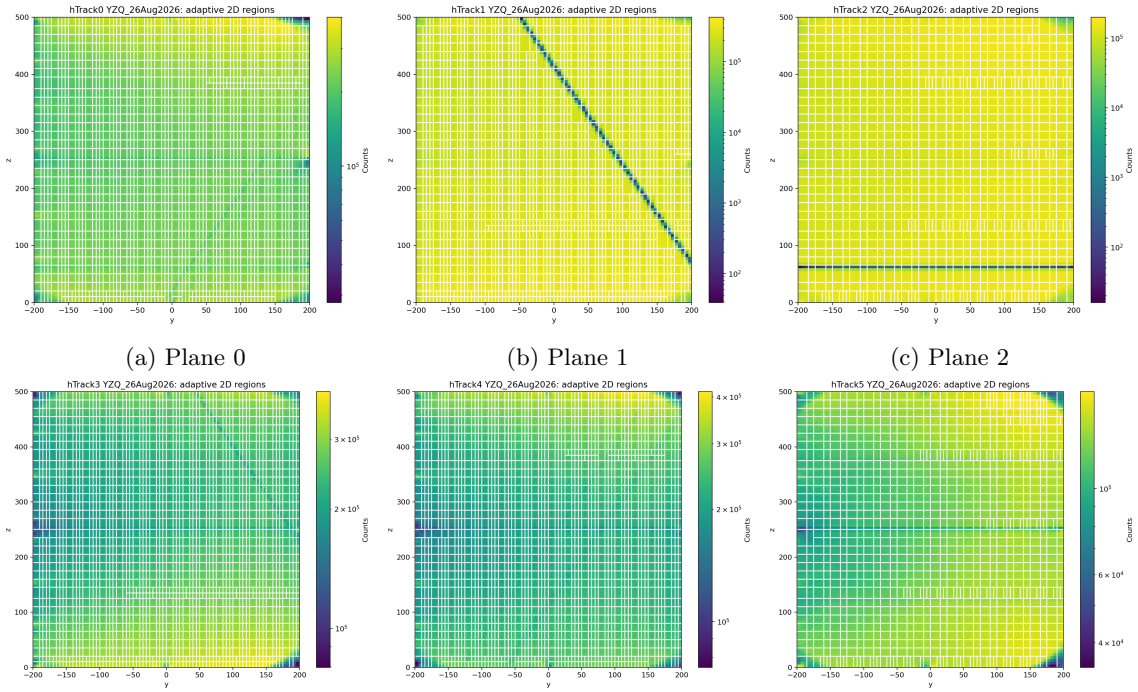


(a) Q, x-bin 1 (b) dQ/dx, x-bin 1 (c) Width, x-bin 1

Figure 11: Example YZ data/MC MPV ratio maps for plane 0, angular region 001, and three response observables.

Appendix A 2D Splines in Gen2

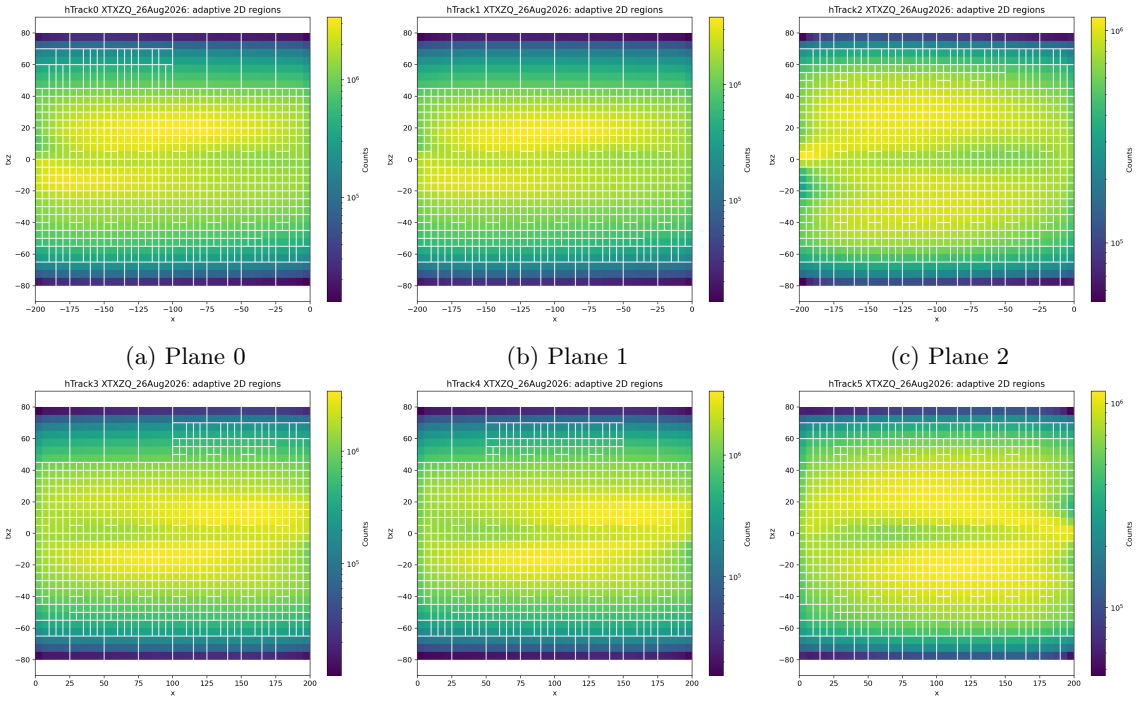
A.1 Adaptive Binning Diagnostics



(a) Plane 0 (b) Plane 1 (c) Plane 2

(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 12: `YZQ` adaptive 2D regions for all planes.

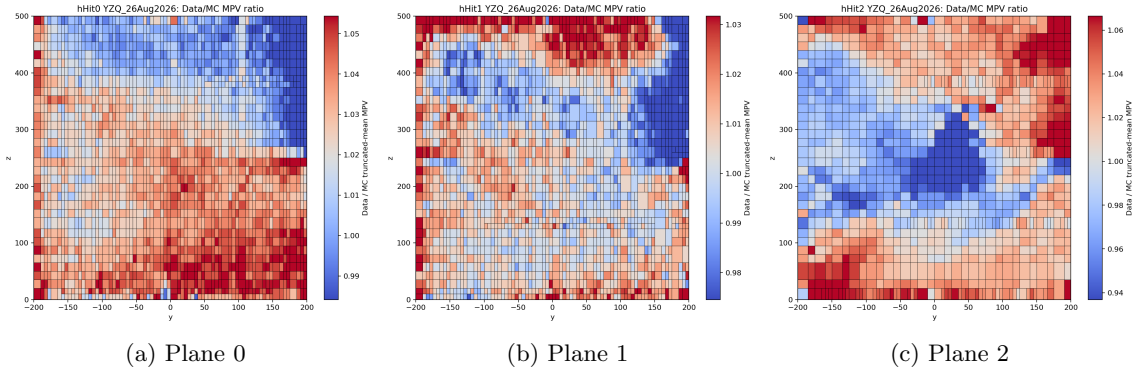


(a) Plane 0 (b) Plane 1 (c) Plane 2

(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 13: `XTXZQ` adaptive 2D regions for all planes.

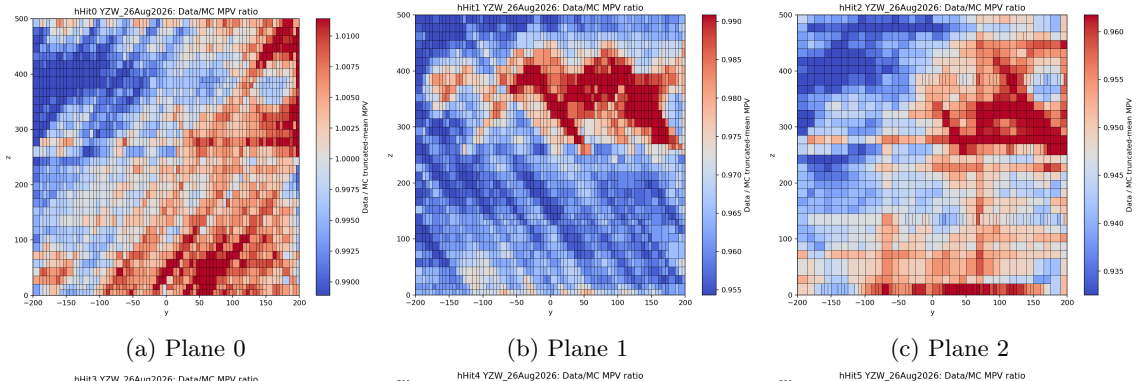
A.2 Ratio Map Diagnostics



(a) Plane 0 (b) Plane 1 (c) Plane 2

(d) Plane 3 (e) Plane 4 (f) Plane 5

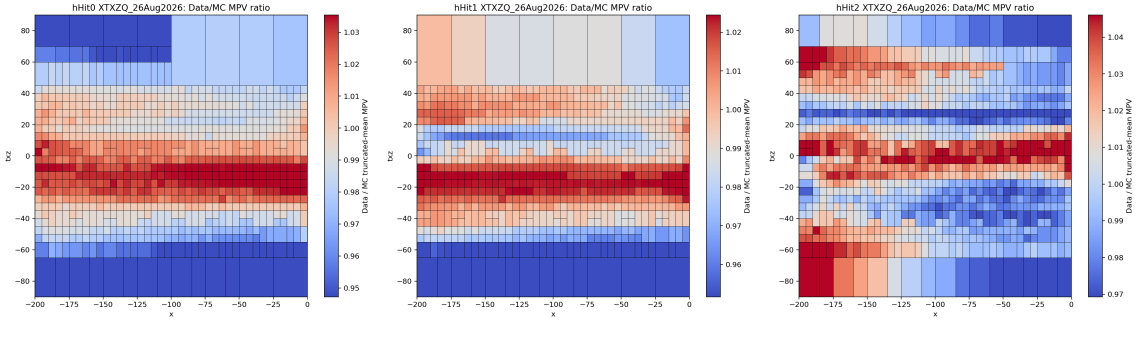
Figure 14: `YZQ` data/MC MPV ratio maps for all planes.



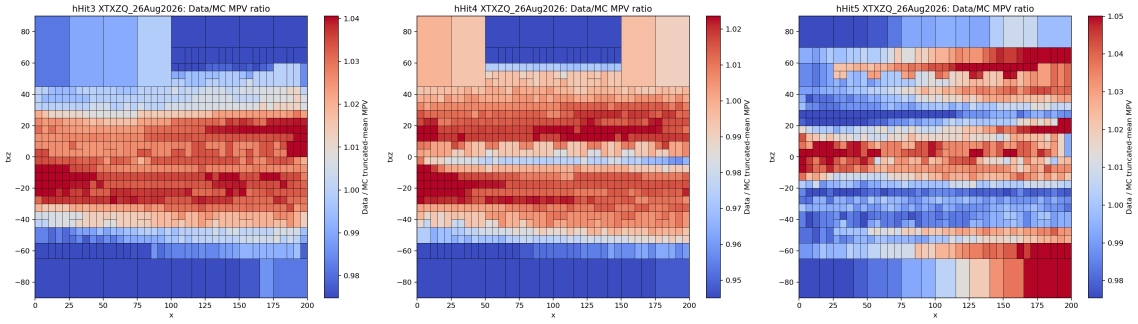
(a) Plane 0 (b) Plane 1 (c) Plane 2

(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 15: `YZW` data/MC MPV ratio maps for all planes.

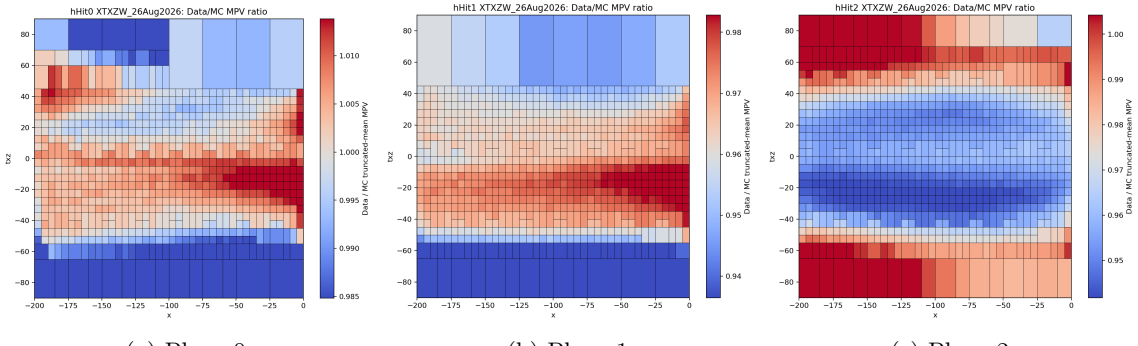


(a) Plane 0 (b) Plane 1 (c) Plane 2

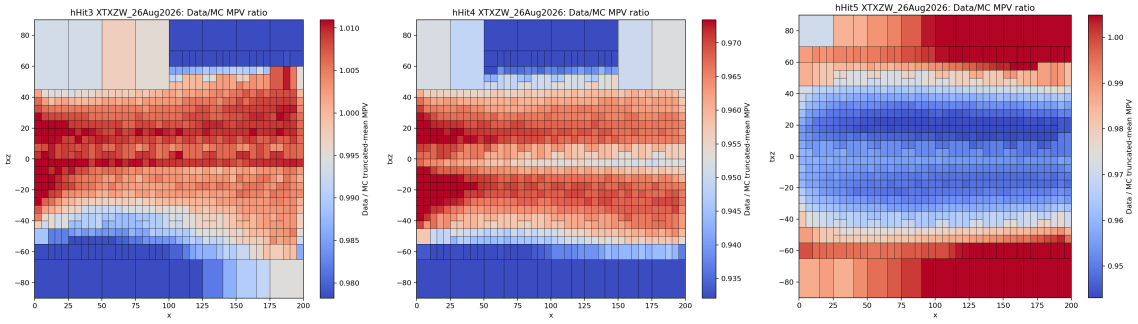


(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 16: `XTXZQ` data/MC MPV ratio maps for all planes.



(a) Plane 0 (b) Plane 1 (c) Plane 2



(d) Plane 3 (e) Plane 4 (f) Plane 5

Figure 17: `XTXZW` data/MC MPV ratio maps for all planes.